

OPTIONAL — Agent Harness + GitHub Copilot Agent

Compare opinionated scaffolding with a separate coding-agent service integration

Source: <https://learn.microsoft.com/en-us/agent-framework/concepts/harness?tabs=python> · +1 in notes

OPTIONAL — Why look now?

No shortcut

Harness does not replace your understanding of sessions, context providers, middleware, or evaluation

Harness shows scaffolding

An opinionated, batteries-included composition

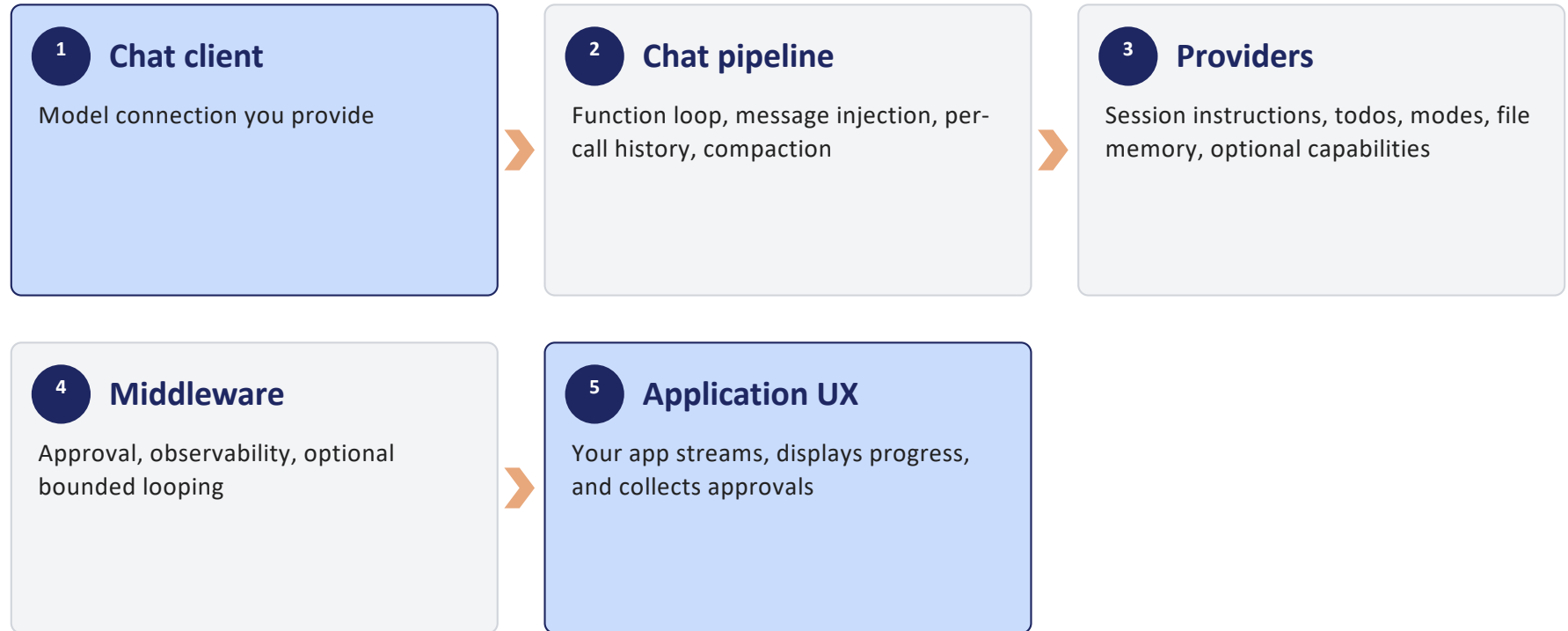
Copilot shows another backend

A coding-oriented agent service/runtime

Optional by design

Neither path replaces the primitives or is required for the Day 3 lab

OPTIONAL — create_harness_agent architecture



OPTIONAL — Capability and release boundaries

Capability	Python harness status
create_harness_agent factory	Released
Todo tracking and plan/execute modes	Enabled by default
File memory	Enabled by default
Skills	Opt-in through provider or paths
Background agents, shared file access, looping	Experimental
Shell tooling	Pre-release agent-framework-tools package

OPTIONAL — GitHub Copilot is a separate agent integration

Harness Agent

- `create_harness_agent(client=chat_client)`
- Opinionated composition over a chat client
- Returns a configured MAF Agent

GitHubCopilotAgent

- Separate MAF agent service integration
- GitHub Copilot SDK owns its runtime/tool loop
- Used directly as a standard MAF Agent

OPTIONAL — GitHub Copilot capabilities

Standard agent operations

Run, stream, and reuse a session

Function tools

Add custom callable tools

Coding runtime

Shell, file operations, and URL fetching when permitted

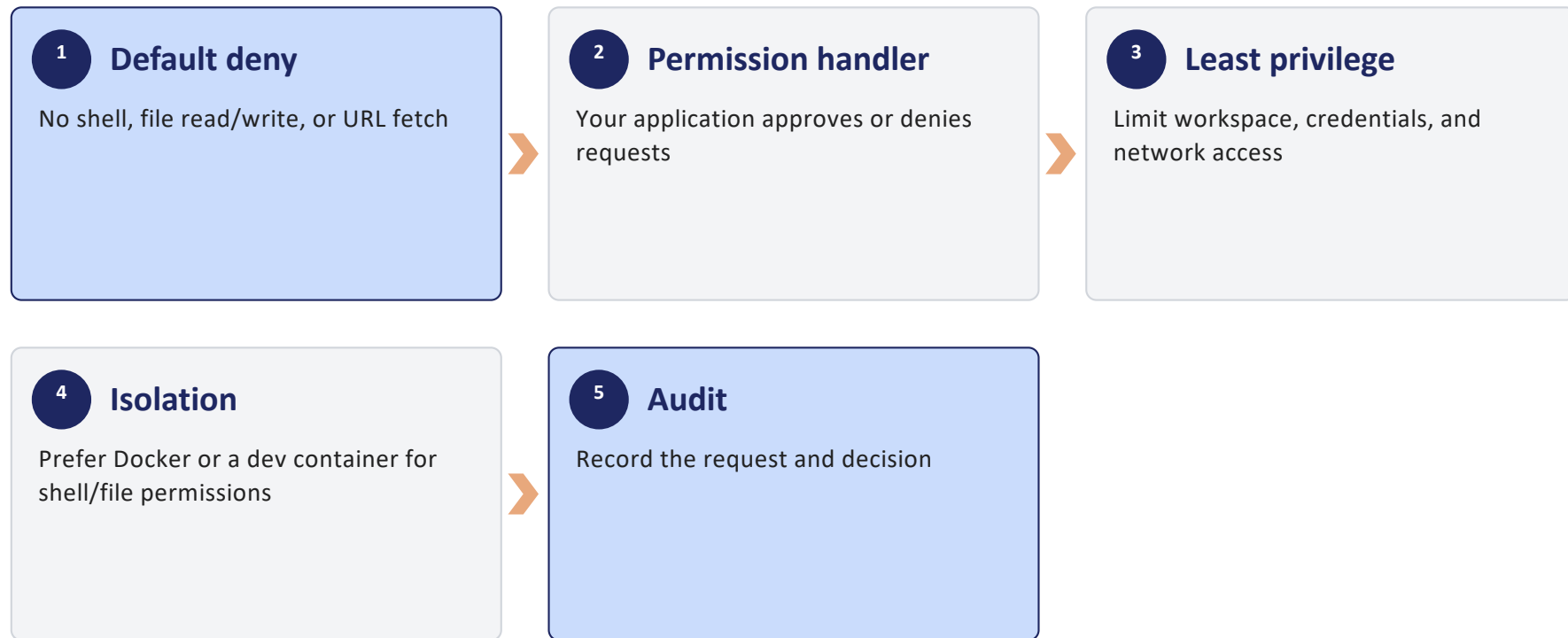
MCP

Local stdio and remote HTTP server configuration

Context providers

Run before and after GitHub Copilot invocations

OPTIONAL — Permissions are a hard boundary



OPTIONAL — No official direct composition pattern

Do not teach

```
create_harness_agent(client=GitHubCopilotAgent(...))
```

Why

The factory contract is a chat client; GitHubCopilotAgent is an Agent

Supported view

Choose one construction path for the application boundary

Future-proofing

Re-check official docs if a composition API is released

OPTIONAL — Choose the least opinionated fit

Choice	Best fit	You still own
Plain Agent	Focused assistant with explicit composition	Every provider, tool, policy, and loop choice
Harness Agent	Long-running work needing opinionated scaffolding	Configuration, storage, permissions, UX, eval
GitHubCopilotAgent	Coding-oriented Copilot runtime capabilities	Permission handler, isolation, tools, eval

OPTIONAL — Takeaways



You treat Harness as released, opinionated MAF scaffolding with experimental edges.



You do not let scaffolding hide sessions, providers, middleware, approvals, or evaluation.



You treat GitHubCopilotAgent as a separate standard MAF agent service integration.



You isolate and explicitly permission shell and file capabilities.



You do not assume an undocumented Harness + GitHubCopilotAgent composition.